Java Programming Notes

Topic: Loops (for, while, do-while, enhanced for, nested loops, break & continue)

1. Introduction to Loops

Loops are used in Java to execute a block of code repeatedly, as long as a given condition is true. They help avoid writing repetitive code and are essential for tasks like iterating over arrays, processing collections, and performing repeated calculations.

Java provides four main types of loops:

- for loop — used when the number of iterations is known
- while loop — used when the number of iterations is unknown, condition-checked first
- do-while loop — similar to while, but executes the body at least once
- enhanced for loop (for-each) — used to iterate over arrays and collections

2. The for Loop

The for loop is ideal when you know in advance how many times you want to execute a statement or a block of statements.

```
Syntax:
for (initialization; condition; update) {
    // code to be executed
}

Example:
for (int i = 1; i <= 5; i++) {
    System.out.println("Count: " + i);
}
// Output: Count: 1, Count: 2, Count: 3, Count: 4, Count: 5
```

initialization runs once before the loop starts. **condition** is checked before every iteration — the loop continues while it is true. **update** runs after each iteration (commonly i++ or i--).

3. The while Loop

The while loop checks the condition before executing the loop body. If the condition is false initially, the loop body never executes.

```
Syntax:
while (condition) {
    // code to be executed
}

Example:
int i = 1;
while (i <= 5) {
```

```
    System.out.println("Count: " + i);
    i++;
}
```

4. The do-while Loop

The do-while loop executes the loop body first and checks the condition afterward. This guarantees the body runs at least once, regardless of the condition.

```
Syntax:
do {
    // code to be executed
} while (condition);

Example:
int i = 1;
do {
    System.out.println("Count: " + i);
    i++;
} while (i <= 5);
```

Note the semicolon after the while(condition) — it is required in do-while but not in a regular while loop.

5. The Enhanced for Loop (for-each)

Used to iterate over arrays or collections without needing an index variable. It is simpler and less error-prone than a standard for loop when you just need to read each element.

```
Syntax:
for (dataType variable : array) {
    // code to be executed
}

Example:
int[] numbers = {10, 20, 30, 40};
for (int num : numbers) {
    System.out.println(num);
}
```

Limitation: you cannot easily access the index, and you cannot modify the original array elements through the loop variable.

6. Nested Loops

A loop inside another loop is called a nested loop. The inner loop completes all its iterations for each single iteration of the outer loop. Commonly used for 2D arrays, grids, and pattern printing.

```
Example (multiplication table):
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.print(i * j + "\t");
    }
    System.out.println();
}

Output:
1   2   3
2   4   6
3   6   9
```

7. break and continue Statements

break — immediately terminates the loop entirely.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;
    }
    System.out.println(i);
}
// Output: 1 2 3 4  (loop stops when i == 5)
```

continue — skips the current iteration and moves to the next one.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    System.out.println(i);
}
// Output: 1 2 4 5  (3 is skipped)
```

8. Labeled Loops

Labels let break and continue target a specific outer loop when working with nested loops.

```
outer:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (j == 2) {
            continue outer;
        }
        System.out.println("i=" + i + " j=" + j);
    }
```

```
}
```

9. Comparison of Loop Types

Loop Type	Condition Check	Guaranteed Execution	Best Used When
for	Before each iteration	No	Number of iterations is known
while	Before each iteration	No	Iterations depend on a condition
do-while	After each iteration	Yes (at least once)	Body must run at least once
for-each	Automatic (per element)	No (skipped if empty)	Iterating arrays/collections

10. Infinite Loops

An infinite loop runs forever because its condition never becomes false. Sometimes used intentionally (e.g., server listening loops) with a break statement to exit.

```
// Intentional infinite loop with break
while (true) {
    // do something
    if (someCondition) {
        break;
    }
}

// Common mistake (forgetting to update i)
for (int i = 0; i < 10; ) {
    System.out.println(i);
    // i never changes -> infinite loop
}
```

11. Key Points to Remember

- Always ensure the loop condition eventually becomes false to avoid infinite loops.
- Use for when the number of iterations is known in advance.
- Use while when looping depends on a condition evaluated before each pass.
- Use do-while when the loop body must execute at least once.
- Use for-each for clean, read-only iteration over arrays and collections.
- break exits the loop completely; continue skips to the next iteration.
- Nested loops multiply iterations: outer runs, and for each outer pass the inner loop runs fully.
- Labeled break/continue give control over which loop is affected in nested structures.